

LAPORAN *PROJECT MACHINE LEARNING*
KLASIFIKASI ONLINE SHOPPERS PURCHASING INTENTION DATASET



Disusun oleh:

1. Faza Hamdi Yogaswara (K1D024011)
2. Nayla Safira Aulia (K1D024012)
3. Naila Dwina Azzahra (K1D024020)
4. Muhammad Alief Badrudin (K1D024030)
5. Rahma Aulia Fatma (K1D024032)

Disusun untuk memenuhi tugas terstruktur:

Mata kuliah : *Machine Learning*

Dosen : Lutfiah Maharani Siniwi, S.Stat., M.Stat.

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI STATISTIKA
UNIVERSITAS JENDERAL SOEDIRMAN
PURWOKERTO

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, kami dapat menyelesaikan laporan *project* akhir mata kuliah *Machine Learning* ini dengan baik dan tepat waktu.

Laporan ini disusun untuk memenuhi tugas *project* pada mata kuliah *Machine Learning*. *Project* ini bertujuan untuk menerapkan seluruh alur kerja *machine learning workflow* secara nyata, mulai dari pendefinisian masalah, eksplorasi data (EDA), prapemrosesan, rekayasa fitur, hingga tahap pemodelan dan evaluasi menggunakan berbagai algoritma klasifikasi.

Dalam proses pengerjaan *project* hingga penyusunan laporan ini, kami banyak mendapatkan bantuan, bimbingan, serta dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini kami ingin menyampaikan terima kasih yang sebesar-besarnya kepada:

1. Ibu Lutfiah Maharani Siniwi, S.Stat., M.Stat., selaku dosen pengampu mata kuliah *machine learning* yang telah memberikan ilmu, arahan, dan bimbingan yang sangat berharga.
2. Seluruh anggota kelompok yang telah bekerja keras dan menunjukkan kontribusi.

Kami menyadari bahwa laporan dan model *machine learning* yang kami kembangkan ini masih jauh dari kesempurnaan. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari dosen pengampu maupun pembaca sekalian demi perbaikan di masa yang akan datang.

Akhir kata, semoga laporan *project* akhir ini dapat memberikan manfaat, menambah wawasan, serta menjadi referensi yang berguna bagi perkembangan implementasi *machine learning* ke depan.

Purwokerto, 08 Juni 2026

Penulis,

Kelompok F

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
DAFTAR TABEL	v
DAFTAR GAMBAR.....	vi
BAB I PENDAHULUAN	1
A. Latar Belakang.....	1
B. Rumusan Masalah.....	2
C. Tujuan.....	2
BAB II TINJAUAN DATASET	3
A. Sumber Dataset.....	3
B. Karakteristik Dataset	3
C. Data <i>Dictionary</i>	4
D. Identifikasi Variabel	5
1. Variabel Dependen (Target)	5
2. Variabel Independen (Fitur)	6
BAB III METODOLOGI	8
A. <i>Exploratory Data Analysis</i> (Eksplorasi Dataset).....	8
1. Distribusi Variabel Target (Revenue).....	8
2. Matriks Korelasi	8
3. Distribusi Variabel Numerik	8
4. Deteksi <i>Outlier</i> melalui <i>Boxplot</i>	9
B. Data <i>Preprocessing</i>	9
1. Encoding Variabel Kategorik	9
2. Analisis <i>Feature Importance</i> dan Seleksi Fitur	10
3. Normalisasi dan Standarisasi Data	10
4. Penanganan Ketidakseimbangan Kelas dengan SMOTE	11
5. Pembagian Data	11
C. <i>Modelling</i>	12
1. Algoritma yang Digunakan	12

2. Metrik Evaluasi.....	13
3. Optimasi <i>Hyperparameter</i>	14
BAB IV HASIL DAN PEMBAHASAN.....	15
A. <i>Feature Importance</i>	15
B. Hasil Pemodelan	17
1. Penanganan Ketidakseimbangan Kelas dengan SMOTE	17
2. Pembagian Data (<i>Split Data</i>)	18
3. Pemodelan Klasifikasi	19
4. Perbandingan Performa Model	21
C. Analisis <i>Overfitting</i> dan <i>Underfitting</i>	22
D. Hasil <i>Hyperparameter Tuning</i>	24
1. Hasil Optimasi <i>Hyperparameter</i> Random Forest	24
2. Hasil Parameter Optimal	25
3. Evaluasi Model Setelah Optimasi.....	26
4. Analisis <i>Learning Curve</i>	27
5. Perbandingan Model Awal dan Model Hasil Tuning.....	29
BAB V PENUTUP	30
A. Kesimpulan.....	30
B. Saran	31
LAMPIRAN	33

DAFTAR TABEL

Tabel 2.1 Tabel Dictionary	4
Tabel 2.2 Variabel Dependen	5
Tabel 2.3 Variabel Independen	6
Tabel 3.1 Algoritma Machine Learning yang Digunakan	12
Tabel 3.2 Metrik Evaluasi Model Klasifikasi	13
Tabel 4.1 Skor Feature Importance 10 Variabel Teratas	16
Tabel 4.2 Perbandingan Distribusi Data Sebelum dan Setelah SMOTE	18
Tabel 4.3 Perbandingan Matriks Evaluasi Lima Model Klasifikasi	21
Tabel 4.4 Tabel Kombinasi Hyperparameter	25
Tabel 4.5 Metrik Evaluasi Model Random Forest Setelah Tuning	26
Tabel 4.6 Perbandingan Matriks Evaluasi Sebelum dan Setelah Tuning	29

DAFTAR GAMBAR

Gambar 4.1 Skor Feature Importance 10 Variabel Teratas	15
Gambar 4.2 Grafik Learning Curve untuk Tuned Random Forest	28

BAB I

PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi informasi dan komunikasi telah mengubah lanskap industri perdagangan secara global. Industri *e-commerce* kini menjadi salah satu pilar ekonomi digital yang tumbuh dengan sangat pesat. Bagi para pelaku bisnis digital, memahami perilaku konsumen saat beraktifitas di *platform* mereka merupakan kunci utama untuk mempertahankan daya saing dan meningkatkan volume penjualan. Namun, salah satu tantangan terbesar yang dihadapi oleh *platform e-commerce* saat ini adalah rendahnya tingkat konversi belanja, di mana sebagian besar pengunjung situs *web* hanya melihat-lihat produk tanpa melakukan transaksi akhir.

Dalam dunia bisnis daring, setiap interaksi pengunjung meninggalkan jejak data digital yang berharga. Indikator seperti jumlah halaman produk yang dilihat (ProductRelated), durasi waktu kunjungan (ProductRelated_Duration), hingga kualitas halaman *web* yang direkam melalui metrik seperti *Bounce Rates* dan *Exit Rates* memiliki korelasi erat terhadap psikologis konsumen. Keberadaan metrik bernilai tinggi seperti PageValues juga sering kali menjadi indikator krusial apakah sebuah halaman *web* berkontribusi langsung pada pendapatan perusahaan. Selain perilaku di dalam situs, keputusan belanja juga kerap dipengaruhi oleh faktor eksternal temporal, seperti momentum hari raya (SpecialDay), fluktuasi bulan tertentu (Month), hingga perbedaan pola belanja di akhir pekan (Weekend).

Untuk mengoptimalkan strategi pemasaran dan meningkatkan pengalaman pengguna secara personal, perusahaan membutuhkan sistem yang mampu mendeteksi niat beli konsumen secara *real-time*. Pendekatan konvensional yang mengandalkan analisis deskriptif pasca transaksi dinilai kurang responsif untuk menangkap dinamika perilaku konsumen yang berubah dengan cepat. Oleh karena itu, penerapan teknologi pembelajaran mesin (*machine learning*) melalui analisis prediktif terhadap berkas data aktivitas konsumen menjadi solusi yang relevan.

Penelitian proyek ini memanfaatkan dataset yang memuat 12.330 sesi kunjungan konsumen. Melalui pemodelan klasifikasi data, karakteristik perilaku dan profil teknis pengunjung akan diekstrak untuk mengenali pola yang membedakan antara

konsumen yang berniat membeli dengan konsumen yang hanya melakukan penjelajahan biasa. Hasil prediksi ini diharapkan dapat membantu pelaku usaha untuk mengambil keputusan strategis, seperti memberikan promo instan yang tepat sasaran atau melakukan optimalisasi antarmuka web pada halaman yang memiliki potensi konversi tinggi.

B. Rumusan Masalah

Berdasarkan pemaparan latar belakang di atas, rumusan masalah yang akan dibahas dalam proyek ini adalah:

1. Bagaimana karakteristik data dan pola perilaku beraktivitas pengunjung platform *e-commerce* berdasarkan dataset?
2. Atribut atau fitur apa saja yang memiliki pengaruh paling signifikan terhadap keputusan akhir konsumen dalam melakukan pembelian (*Revenue*)?
3. Bagaimana mengimplementasikan dan mengukur performa model *machine learning* terbaik untuk memprediksi intensi pembelian konsumen secara akurat?

C. Tujuan

Adapun tujuan yang ingin dicapai melalui pelaksanaan proyek data sains ini adalah:

1. Untuk menganalisis dan mendeskripsikan karakteristik data serta pola kebiasaan pengunjung *platform e-commerce* saat berinteraksi di dalam situs berdasarkan dataset.
2. Untuk mengidentifikasi dan menentukan atribut atau fitur-fitur utama yang memiliki pengaruh paling signifikan terhadap keputusan akhir konsumen dalam melakukan pembelian (*Revenue*).
3. Untuk mengimplementasikan dan mengukur performa model *machine learning* terbaik guna memprediksi intensi pembelian konsumen secara akurat.

BAB II

TINJAUAN DATASET

A. Sumber Dataset

Dataset yang digunakan dalam penelitian ini adalah “Online Shoppers Purchasing Intention Dataset”. Dataset tersebut diperoleh melalui platform Kaggle yang menyediakan berbagai dataset untuk kebutuhan analisis data dan *machine learning*. Dataset ini berasal dari penelitian mengenai perilaku pengunjung situs *e-commerce* dan telah banyak digunakan dalam studi klasifikasi serta prediksi keputusan pembelian pelanggan.

Dataset ini berisi data aktivitas pengunjung situs *e-commerce* selama satu tahun. Setiap baris data merepresentasikan satu sesi kunjungan pengguna, sedangkan variabel yang tersedia menggambarkan karakteristik perilaku pengguna selama mengakses *website*. Tujuan utama penggunaan dataset ini adalah untuk memprediksi apakah seorang pengunjung akan melakukan pembelian produk yang ditunjukkan oleh variabel target Revenue.

B. Karakteristik Dataset

Dataset Online Shoppers Intention terdiri dari 12.330 baris dan 18 kolom. Berdasarkan hasil `df.info()`, seluruh variabel memiliki 12.330 nilai non-null, artinya tidak ada *missing value* sehingga data dapat langsung digunakan tanpa proses penanganan data hilang terlebih dahulu. Dari sisi tipe data, dataset memuat empat jenis yang berbeda: 7 variabel bertipe `int64`, 7 variabel bertipe `float64`, 2 variabel kategorik bertipe `object`, dan 2 variabel bertipe `bool`. Variabel bertipe `int64` meliputi `Administrative`, `Informational`, `ProductRelated`, `OperatingSystems`, `Browser`, `Region`, dan `TrafficType`, yang merepresentasikan jumlah halaman yang dikunjungi beserta atribut teknis sesi pengguna. Variabel bertipe `float64` terdiri dari `Administrative_Duration`, `Informational_Duration`, `ProductRelated_Duration`, `BounceRates`, `ExitRates`, `PageValues`, dan `SpecialDay`, yang mencerminkan durasi kunjungan dan metrik perilaku pengguna di situs. Variabel kategorik `Month` dan `VisitorType` masing-masing mencatat bulan kunjungan dan jenis pengunjung, sedangkan variabel boolean `Weekend` dan `Revenue` menandai apakah sesi terjadi di akhir pekan dan apakah berujung pada transaksi pembelian.

Karena algoritma *machine learning* umumnya hanya menerima input numerik, variabel kategorik dan boolean perlu dikonversi melalui proses *encoding* sebelum masuk ke tahap pemodelan. Dari segi ukuran, dataset hanya memerlukan sekitar 1,5 MB memori, sehingga cukup ringan untuk diproses.

C. Data Dictionary

Data *dictionary* berfungsi untuk memberikan definisi operasional dari setiap atribut yang terdapat di dalam dataset.

Tabel 2.1 Tabel Dictionary

Variabel	Tipe Data	Deskripsi
Administrative	Numerik (Diskrit)	Jumlah halaman bertipe administratif (seperti akun, keranjang, atau checkout) yang dikunjungi dalam satu sesi.
Administrative_Duration	Numerik (Kontinu)	Total durasi waktu (dalam detik) yang dihabiskan pengguna pada kelompok halaman administratif.
Informational	Numerik (Diskrit)	Jumlah halaman bertipe informasi (seperti profil perusahaan atau kontak) yang dikunjungi dalam satu sesi.
Informational_Duration	Numerik (Kontinu)	Total durasi waktu (dalam detik) yang dihabiskan pengguna pada kelompok halaman informasi.
ProductRelated	Numerik (Diskrit)	Jumlah halaman yang berkaitan langsung dengan produk (seperti katalog atau detail barang).
ProductRelated_Duration	Numerik (Kontinu)	Total durasi waktu (dalam detik) yang dihabiskan pengguna pada kelompok halaman produk.
BounceRates	Numerik (Kontinu)	Persentase pengunjung yang masuk ke suatu halaman lalu langsung keluar (bounce) tanpa interaksi lanjutan.
ExitRates	Numerik (Kontinu)	Persentase rasio kunjungan yang berakhir pada halaman spesifik tersebut dari total kunjungan ke halaman itu.
PageValues	Numerik (Kontinu)	Nilai rata-rata halaman web yang mengukur kontribusinya terhadap keberhasilan transaksi finansial.

Variabel	Tipe Data	Deskripsi
SpecialDay	Numerik (Kontinu)	Parameter kedekatan waktu kunjungan dengan hari-hari khusus berskala besar (misal: Valentine, Hari Ibu).
Month	Kategorikal (String)	Bulan terjadinya sesi kunjungan dalam bentuk singkatan teks bahasa Inggris.
OperatingSystems	Kategorikal (Integer)	Kode indeks numerik yang mewakili jenis Sistem Operasi (OS) perangkat pengunjung.
Browser	Kategorikal (Integer)	Kode indeks numerik yang mewakili jenis peramban (browser) yang digunakan pengunjung.
Region	Kategorikal (Integer)	Kode indeks wilayah geografis atau asal lokasi jaringan pengunjung berada.
TrafficType	Kategorikal (Integer)	Kode indeks numerik yang menunjukkan asal/sumber kedatangan lalu lintas web (iklan, organik, dll).
VisitorType	Kategorikal (String)	Klasifikasi tipe pengunjung berdasarkan histori kunjungan ke platform.
Weekend	Kategorikal (Boolean)	Status biner yang menandakan apakah sesi belanja terjadi di akhir pekan.
Revenue	Kategorikal (Boolean)	Menandakan apakah sesi kunjungan tersebut berhasil menghasilkan pendapatan/pembelian

D. Identifikasi Variabel

Untuk keperluan pemodelan prediktif berbasis pembelajaran mesin, seluruh komponen variabel di dalam dataset dipilah ke dalam peran fungsionalnya masing-masing sebagai berikut:

1. Variabel Dependen (Target)

Variabel dependen merupakan variabel utama yang nilainya dipengaruhi oleh variabel lain dan menjadi fokus prediksi akhir dalam penelitian ini.

Tabel 2.2 Variabel Dependen

Variabel	Skala Pengukuran	Peran dalam Model
Revenue	Nominal (Biner)	Variabel Target. Berfungsi memisahkan sesi yang menghasilkan

		transaksi (TRUE) dengan sesi yang tidak menghasilkan transaksi (FALSE).
--	--	---

2. Variabel Independen (Fitur)

Variabel independen adalah faktor-faktor penjelas (fitur) yang digunakan oleh model untuk mengekstrak pola perilaku konsumen guna memprediksi nilai target. Variabel ini dibagi menjadi beberapa kelompok dimensi sebagai berikut:

Tabel 2.3 Variabel Independen

Variabel	Skala Pengukuran	Peran dalam Model
Administrative	Rasio / Diskrit	Mengukur intensitas halaman kelola akun atau <i>checkout</i> yang dikunjungi pengguna.
Administrative_Duration	Rasio / Kontinu	Mengukur total durasi waktu (detik) yang dihabiskan pada halaman administratif.
Informational	Rasio / Diskrit	Mengukur intensitas halaman informasi (kontak/regulasi) yang dikunjungi pengguna.
Informational_Duration	Rasio / Kontinu	Mengukur total durasi waktu (detik) yang dihabiskan pada halaman informasi.
ProductRelated	Rasio / Diskrit	Mengukur intensitas halaman katalog dan detail produk yang dikunjungi pengguna.
ProductRelated_Duration	Rasio / Kontinu	Mengukur total durasi waktu (detik) yang dihabiskan pada halaman produk.
BounceRates	Rasio / Kontinu	Mengukur persentase pengguna yang langsung meninggalkan situs dari suatu halaman tanpa interaksi.
ExitRates	Rasio / Kontinu	Mengukur persentase rasio kunjungan yang berakhir (keluar dari situs) pada halaman tersebut.
PageValues	Rasio / Kontinu	Mengukur nilai rata-rata kontribusi suatu halaman terhadap keberhasilan konversi transaksi.
SpecialDay	Rasio / Kontinu	Mengukur kedekatan hari kunjungan dengan momen hari raya atau perayaan besar tertentu.

Variabel	Skala Pengukuran	Peran dalam Model
Month	Nominal	Mengidentifikasi pengaruh tren belanja bulanan atau musiman sepanjang tahun.
Weekend	Nominal (Biner)	Mengidentifikasi perbedaan pola perilaku belanja antara hari kerja biasa dengan akhir pekan.
OperatingSystems	Nominal	Mengklasifikasikan jenis sistem operasi perangkat yang digunakan oleh pengunjung.
Browser	Nominal	Mengklasifikasikan jenis peramban (browser) yang digunakan pengunjung saat mengakses situs.
Region	Nominal	Mengklasifikasikan asal wilayah geografis atau lokasi jaringan internet pengunjung.
TrafficType	Nominal	Mengklasifikasikan sumber atau saluran kedatangan pengunjung ke situs web (iklan, organik, dll.).
VisitorType	Nominal	Membedakan karakteristik antara pelanggan baru, pelanggan lama (returning), atau kategori lainnya.

BAB III

METODOLOGI

A. *Exploratory Data Analysis* (Eksplorasi Dataset)

Tahap *exploratory data analysis* (EDA) dilakukan untuk memahami karakteristik dan pola dalam dataset sebelum masuk ke tahap pemodelan. EDA pada penelitian ini mencakup analisis distribusi variabel target, pemeriksaan korelasi antarvariabel, analisis distribusi variabel numerik, serta deteksi *outlier* melalui visualisasi *boxplot*.

1. Distribusi Variabel Target (Revenue)

Distribusi variabel target Revenue menunjukkan ketidakseimbangan kelas (*class imbalance*) yang cukup besar. Dari 12.330 observasi, 10.422 data (84,53%) masuk kelas Revenue = 0 (tidak membeli), sementara hanya 1.908 data (15,47%) yang masuk kelas Revenue = 1 (melakukan pembelian). Kondisi ini membuat model berpotensi condong memprediksi kelas mayoritas, sehingga pada tahap *preprocessing* diperlukan teknik penyeimbangan data menggunakan SMOTE (*Synthetic Minority Over-sampling Technique*).

2. Matriks Korelasi

Analisis korelasi dilakukan terhadap seluruh variabel numerik untuk melihat hubungan linear antarvariabel. Korelasi positif terkuat ada di antara BounceRates dan ExitRates (0,91), serta antara ProductRelated dan ProductRelated_Duration (0,86). Dua pasangan ini berpotensi memunculkan multikolinearitas yang perlu diperhatikan saat pemodelan. Terhadap Revenue, PageValues punya hubungan positif paling kuat (0,49), disusul ProductRelated (0,16) dan ProductRelated_Duration (0,15). Sebaliknya, ExitRates (-0,21) dan BounceRates (-0,15) berkorelasi negatif semakin tinggi tingkat keluar pengunjung, semakin kecil peluang transaksi terjadi.

3. Distribusi Variabel Numerik

Histogram terhadap 14 variabel numerik menunjukkan bahwa hampir semua variabel terdistribusi miring ke kanan (*right skewed*): Administrative, Administrative_Duration, Informational, Informational_Duration, ProductRelated, ProductRelated_Duration, BounceRates, dan PageValues. Artinya sebagian besar pengunjung beraktivitas rendah di situs, dengan segelintir pengunjung yang berperilaku sangat intensif. SpecialDay didominasi nilai 0 kunjungan yang terjadi

menjelang hari promosi sangat sedikit. Variabel `OperatingSystems`, `Browser`, `Region`, dan `TrafficType` menunjukkan dominasi pada kategori-kategori tertentu dengan distribusi yang tidak merata.

Saat histogram dipecah berdasarkan kelas `Revenue`, `PageValues` menunjukkan perbedaan distribusi paling mencolok. Pengunjung dengan `Revenue = True` cenderung punya nilai `PageValues` lebih tinggi dan mengakses lebih banyak halaman produk (`ProductRelated`) dengan durasi lebih lama. Nilai `BounceRates` dan `ExitRates` yang tinggi justru lebih banyak muncul pada kelas `Revenue (False)`.

4. Deteksi *Outlier* melalui *Boxplot*

Boxplot terhadap 10 variabel numerik utama memperlihatkan bahwa semua variabel punya *outlier* dalam jumlah cukup besar di sisi atas distribusi. `ProductRelated_Duration` adalah yang paling ekstrem, dengan beberapa observasi melampaui 60.000 detik. `PageValues`, `Informational`, dan `BounceRates` menunjukkan pola serupa: mayoritas data menumpuk di nilai rendah, tapi ada sejumlah kecil observasi dengan nilai sangat tinggi. Keberadaan *outlier* ini sejalan dengan distribusi *right skewed* yang ditemukan di tahap histogram, dan menjadi pertimbangan dalam memilih metode normalisasi pada tahap *preprocessing*.

B. Data Preprocessing

Tahap *preprocessing* merupakan serangkaian proses transformasi data yang dilakukan untuk mempersiapkan dataset agar siap digunakan dalam pembangunan model *machine learning*. Pada penelitian ini, proses *preprocessing* mencakup encoding variabel kategorik, seleksi fitur, normalisasi dan standarisasi data, serta penanganan ketidakseimbangan kelas menggunakan SMOTE.

1. Encoding Variabel Kategorik

Dataset Online Shoppers Intention mengandung dua variabel kategorik bertipe object, yaitu `Month` dan `VisitorType`, serta dua variabel bertipe boolean, yaitu `Weekend` dan `Revenue`. Agar seluruh variabel dapat diproses oleh algoritma machine learning, dilakukan proses konversi tipe data sebagai berikut.

Variabel `Weekend` dan `Revenue` yang bertipe boolean dikonversi menjadi representasi numerik biner (0 dan 1) menggunakan `LabelEncoder` dari library `Scikit-Learn`. Sementara itu, variabel `Month` dan `VisitorType` yang bersifat nominal dikonversi menggunakan metode `One-Hot Encoding` melalui fungsi

pd.get_dummies(). Variabel Month menghasilkan 10 variabel dummy baru (Month_Aug, Month_Dec, Month_Feb, Month_Jul, Month_June, Month_Mar, Month_May, Month_Nov, Month_Oct, Month_Sep), sedangkan VisitorType menghasilkan 3 variabel dummy (VisitorType_New_Visitor, VisitorType_Other, VisitorType_Returning_Visitor). Setelah proses encoding, jumlah variabel meningkat dari 18 menjadi 29 variabel dan seluruh variabel telah berada dalam format numerik.

2. Analisis *Feature Importance* dan Seleksi Fitur

Sebelum proses normalisasi dan pembangunan model, dilakukan analisis feature importance menggunakan model XGBoost Classifier untuk mengidentifikasi fitur-fitur yang paling berkontribusi dalam memprediksi variabel target Revenue. Berdasarkan grafik *feature importance*, sepuluh fitur dengan kontribusi tertinggi adalah ProductRelated_Duration (skor 403), ExitRates (402), ProductRelated (344), Administrative_Duration (314), PageValues (271), BounceRates (239), Administrative (189), TrafficType (165), Informational_Duration (162), dan Region (131).

Berdasarkan hasil analisis tersebut, dilakukan seleksi fitur dengan menghapus variabel Browser dan OperatingSystems yang memiliki nilai importance relatif rendah. Variabel Region dan TrafficType juga dihapus sebagai bagian dari proses reduksi kompleksitas model. Sementara itu, variabel-variabel Month tetap dipertahankan karena merepresentasikan faktor musiman (*seasonality*) yang secara teoritis berpotensi memengaruhi perilaku pembelian konsumen. Setelah proses seleksi fitur, jumlah variabel prediktor yang digunakan dalam pemodelan menjadi 24 fitur.

3. Normalisasi dan Standarisasi Data

Proses transformasi skala data dilakukan dalam dua tahap. Pertama, normalisasi menggunakan Min-Max Scaling diterapkan pada lima variabel utama, yaitu ProductRelated, ProductRelated_Duration, PageValues, BounceRates, dan ExitRates. Metode ini mengubah nilai setiap variabel ke dalam rentang 0 hingga 1 sehingga tidak terdapat variabel yang mendominasi variabel lain akibat perbedaan skala.

Kedua, standarisasi menggunakan metode Z-score diterapkan pada sepuluh variabel numerik kontinu, yaitu *Administrative*, *Administrative_Duration*, *Informational*, *Informational_Duration*, *ProductRelated*, *ProductRelated_Duration*, *BounceRates*, *ExitRates*, *PageValues*, dan *SpecialDay*. Proses standarisasi dilakukan dengan mengurangi setiap nilai pengamatan terhadap nilai rata-rata variabelnya, kemudian membaginya dengan simpangan baku variabel tersebut. Melalui proses ini, data ditransformasikan sehingga memiliki rata-rata yang mendekati 0 dan standar deviasi yang mendekati 1. Standarisasi bertujuan untuk menyamakan skala antarvariabel tanpa menghilangkan karakteristik distribusi data yang dimiliki. Dengan demikian, setiap variabel dapat memberikan kontribusi yang lebih proporsional dalam proses pembelajaran model. Transformasi ini sangat penting terutama bagi algoritma yang sensitif terhadap perbedaan skala data, seperti Logistic Regression dan *Support Vector Machine* (SVM), karena dapat membantu meningkatkan stabilitas proses pelatihan serta menghasilkan performa prediksi yang lebih optimal.

4. Penanganan Ketidakseimbangan Kelas dengan SMOTE

Berdasarkan hasil EDA yang telah dilakukan sebelumnya, dataset mengalami ketidakseimbangan kelas yang signifikan dengan proporsi 84,53% untuk kelas 0 dan 15,47% untuk kelas 1. Untuk mengatasi permasalahan ini, diterapkan metode *Synthetic Minority Over-sampling Technique* (SMOTE) dari library *imbalanced-learn*. SMOTE bekerja dengan cara membangkitkan sampel sintetis baru untuk kelas minoritas berdasarkan interpolasi linear antara titik-titik data yang sudah ada di ruang fitur.

Penerapan SMOTE dilakukan secara eksklusif hanya pada data training setelah proses pembagian data (*train-test split*). Hal ini bertujuan untuk mencegah terjadinya *data leakage*, yaitu kondisi di mana informasi dari data testing secara tidak sengaja masuk ke dalam proses pelatihan model sehingga evaluasi menjadi tidak objektif. Setelah SMOTE diterapkan, distribusi kelas pada data training menjadi seimbang sempurna dengan masing-masing kelas memiliki 8.367 sampel (50,0%), sementara data testing tetap dipertahankan pada distribusi aslinya.

5. Pembagian Data

Pembagian dataset menjadi data training dan data testing dilakukan menggunakan fungsi *train_test_split* dari library *Scikit-Learn* dengan proporsi 80% untuk data training dan 20% untuk data testing. Proses pembagian menggunakan parameter *random_state* = 42 untuk memastikan hasil dapat direproduksi, serta parameter *stratify* untuk menjaga proporsi kelas tetap terwakili secara proporsional pada kedua subset data.

Setelah penerapan SMOTE pada data training, dilakukan pembagian ulang terhadap data hasil *resampling* dengan proporsi yang sama (80:20) menggunakan parameter *stratify*. Hasil akhir pembagian data adalah 13.387 sampel untuk data training dan 3.347 sampel untuk data testing, masing-masing dengan 24 fitur prediktor.

C. Modelling

Tahap pemodelan merupakan inti dari penelitian ini, di mana berbagai algoritma klasifikasi *machine learning* dibangun, dilatih, dan dievaluasi secara komparatif. Seluruh proses pemodelan menggunakan data training yang telah melalui tahap preprocessing lengkap, dan dievaluasi menggunakan data *testing* yang tidak pernah dilihat oleh model selama proses pelatihan.

1. Algoritma yang Digunakan

Pada penelitian ini, digunakan lima algoritma klasifikasi yang beragam untuk memperoleh perbandingan performa yang komprehensif. Kelima algoritma tersebut dipilih karena mewakili pendekatan yang berbeda dalam menyelesaikan permasalahan klasifikasi, mulai dari model linier, pohon keputusan tunggal, metode ensemble, hingga metode berbasis kernel. Ringkasan algoritma yang digunakan disajikan pada tabel berikut.

Tabel 3.1 Algoritma Machine Learning yang Digunakan

Algoritma	Tipe	Konfigurasi Utama
Random Forest	Ensemble (Bagging)	<i>random_state</i> =42
Logistic Regression	Linear Classifier	<i>solver</i> ='liblinear', <i>random_state</i> =42
XGBoost	Ensemble (Boosting)	<i>eval_metric</i> ='logloss', <i>random_state</i> =42
Decision Tree	Tree-based	<i>random_state</i> =42
SVM	Kernel-based	<i>kernel</i> ='rbf', <i>probability</i> =True,

Algoritma	Tipe	Konfigurasi Utama
		random_state=42

Kelima algoritma tersebut diinisialisasi dan dilatih menggunakan sintaks yang seragam melalui antarmuka Scikit-Learn, kecuali XGBoost yang menggunakan library xgboost dengan antarmuka yang kompatibel dengan Scikit-Learn. Seluruh model dilatih pada data training yang sama dan dievaluasi pada data testing yang sama untuk memastikan perbandingan yang adil dan konsisten.

2. Metrik Evaluasi

Evaluasi performa setiap model dilakukan menggunakan empat metrik utama yang umum digunakan dalam permasalahan klasifikasi biner. Penjelasan masing-masing metrik disajikan pada Tabel berikut.

Tabel 3.2 Metrik Evaluasi Model Klasifikasi

Metrik	Deskripsi
<i>Accuracy</i>	Proporsi prediksi yang benar dari seluruh observasi
<i>Precision</i>	Proporsi prediksi positif yang benar-benar positif ($True\ Positive / (True\ Positive + False\ Positive)$)
<i>Recall</i>	Proporsi data positif aktual yang berhasil terdeteksi model ($True\ Positive / (True\ Positive + False\ Negative)$)
<i>F1-Score</i>	Rata-rata harmonik antara <i>Precision</i> dan <i>Recall</i> sebagai metrik keseimbangan performa

Di antara keempat metrik tersebut, Recall pada kelas 1 (Revenue = *True*) menjadi metrik yang paling diprioritaskan dalam penelitian ini. Hal ini dikarenakan dalam konteks bisnis *e-commerce*, kesalahan model dalam mengklasifikasikan pengunjung yang sebenarnya akan melakukan pembelian sebagai "tidak akan membeli" (*false negative*) memiliki dampak yang lebih merugikan dibandingkan kesalahan sebaliknya, karena perusahaan akan kehilangan kesempatan untuk memberikan intervensi pemasaran yang tepat sasaran kepada calon pembeli potensial.

Evaluasi dilakukan secara terpisah untuk data *training* dan data *testing* guna mendeteksi kemungkinan adanya *overfitting*, yaitu kondisi di mana model memiliki performa yang sangat tinggi pada data *training* namun mengalami penurunan signifikan pada data *testing*. Hasil evaluasi lengkap disajikan melalui *classification report* yang mencakup nilai *precision*, *recall*, dan *F1-score* untuk

masing-masing kelas, serta akurasi keseluruhan (*macro average* dan *weighted average*).

3. Optimasi *Hyperparameter*

Setelah pemilihan model terbaik dari perbandingan lima algoritma, dilakukan proses optimasi hyperparameter menggunakan metode *RandomizedSearchCV* dari library *Scikit-Learn*. Berbeda dengan *GridSearchCV* yang memeriksa seluruh kombinasi parameter secara menyeluruh, *RandomizedSearchCV* bekerja dengan mengambil sampel kombinasi parameter secara acak dari distribusi yang telah ditentukan. Pendekatan ini dipilih karena lebih efisien dari segi waktu komputasi, terutama ketika ruang pencarian parameter sangat besar.

Parameter yang dioptimasi untuk algoritma *Random Forest* mencakup empat parameter utama, yaitu: (1) jumlah pohon yang dibangun (*n_estimators*) dengan rentang 100 hingga 500 pohon; (2) batas kedalaman maksimum pohon (*max_depth*) dengan opsi tanpa batasan hingga kedalaman 50; (3) jumlah minimal sampel untuk membagi cabang (*min_samples_split*) antara 2 hingga 20; serta (4) jumlah minimal sampel pada setiap daun akhir (*min_samples_leaf*) antara 1 hingga 10.

Proses *RandomizedSearchCV* dijalankan sebanyak 100 iterasi eksperimen acak (*n_iter* = 100), di mana setiap eksperimen dievaluasi menggunakan metode *5-fold cross-validation*, sehingga secara total terdapat 500 kali proses pelatihan model. Kualitas setiap kombinasi parameter dinilai berdasarkan akurasi *cross-validation*. Untuk mempercepat komputasi, seluruh proses dijalankan secara paralel menggunakan seluruh inti komputer yang tersedia (*n_jobs* = -1). Model terbaik yang diperoleh kemudian dievaluasi kembali menggunakan data testing untuk dibandingkan dengan model awal sebelum *tuning*.

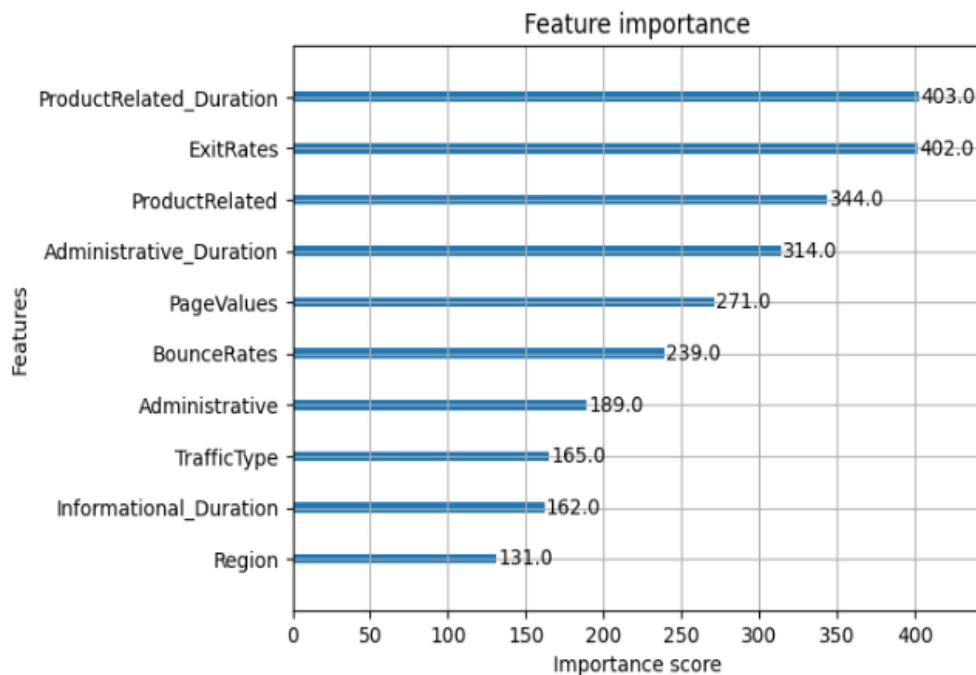
BAB IV

HASIL DAN PEMBAHASAN

A. *Feature Importance*

Pada tahap ini, dilakukan analisis *feature importance* untuk mengidentifikasi variabel prediktor yang memiliki pengaruh paling signifikan terhadap variabel target, yaitu Revenue (keputusan pembelian). Analisis ini bertujuan untuk memahami faktor-faktor perilaku pengunjung *website* yang paling menentukan keberhasilan transaksi, sekaligus menjadi dasar untuk proses seleksi fitur guna mengurangi kompleksitas model tanpa mengorbankan akurasi prediksi.

Analisis *feature importance* dilakukan dengan membangun model XGBoost Classifier pada data latih. Metode ini dipilih karena algoritma XGBoost secara intrinsik menyediakan skor kepentingan fitur berdasarkan seberapa sering suatu variabel digunakan untuk membangun pohon keputusan dalam proses *boosting*. Hasil analisis disajikan pada gambar dan tabel di bawah ini.



Gambar 4.1 Skor Feature Importance 10 Variabel Teratas

Berdasarkan visualisasi pada Gambar 4.1, berikut adalah peringkat variabel berdasarkan kontribusinya terhadap prediksi:

Tabel 4.1 Skor Feature Importance 10 Variabel Teratas

Peringkat	Nama Variabel	Skor Importance
1	ProductRelated_Duration	403
2	ExitRates	402
3	ProductRelated	344
4	Administrative_Duration	314
5	PageValues	271
6	BounceRates	239
7	Administrative	189
8	TrafficType	165
9	Informational_Duration	162
10	Region	131

Berdasarkan hasil yang diperoleh, terdapat beberapa poin penting yang dapat diinterpretasikan. Pertama, tiga variabel dengan skor *importance* tertinggi secara konsisten berasal dari kategori interaksi pengunjung dengan halaman produk, yaitu ProductRelated_Duration, ExitRates, dan ProductRelated. Hal ini mengindikasikan bahwa durasi yang dihabiskan untuk melihat produk, jumlah halaman produk yang dikunjungi, dan tingkat keluarnya pengunjung dari situs merupakan faktor paling krusial dalam membedakan pengunjung yang akan melakukan pembelian dengan yang tidak.

Kedua, variabel PageValues menempati peringkat kelima dengan skor 271. Hal ini menunjukkan bahwa nilai halaman yang dikunjungi pengunjung memiliki hubungan yang erat dengan potensi transaksi. Pengunjung dengan PageValues yang tinggi secara signifikan lebih mungkin untuk melakukan pembelian.

Ketiga, variabel seperti TrafficType dan Region masih memiliki kontribusi terhadap prediksi, tetapi nilainya relatif lebih rendah dibandingkan faktor perilaku. Hal ini menunjukkan bahwa meskipun sumber lalu lintas pengunjung dan wilayah asal dapat menjadi pertimbangan, faktor-faktor tersebut tidak sekuat perilaku navigasi pengunjung dalam menentukan keputusan pembelian.

Keempat, variabel seperti Browser, OperatingSystems, dan Region memiliki skor *importance* di bawah 20 poin. Temuan ini menjadi dasar untuk melakukan seleksi fitur, di mana variabel-variabel dengan kontribusi rendah dapat dihapus untuk menyederhanakan model tanpa mengurangi akurasi prediksi secara signifikan.

Berdasarkan hasil analisis di atas, dilakukan proses seleksi fitur dengan tujuan mengurangi *overfitting*, mempercepat waktu komputasi, dan mempertahankan performa model. Variabel yang di-drop meliputi Browser, OperatingSystems, Region, dan TrafficType. Keputusan ini didasarkan pada skor *importance* yang relatif rendah, di mana keempat variabel tersebut memiliki kontribusi yang kecil terhadap prediksi sehingga pengeluarannya tidak akan menurunkan akurasi model secara signifikan.

Sementara itu, variabel Month tetap dipertahankan meskipun tidak muncul dalam 10 besar *importance*. Keputusan ini diambil karena faktor musiman secara teoritis memiliki pengaruh signifikan terhadap perilaku konsumen dan permintaan pasar, sehingga kehadirannya dalam model dianggap penting untuk menangkap pola periodik dari data. Dengan demikian, proses seleksi fitur ini menghasilkan model yang lebih sederhana, efisien, dan tetap mempertahankan kemampuan prediktif yang optimal.

B. Hasil Pemodelan

1. Penanganan Ketidakseimbangan Kelas dengan SMOTE

Sebelum proses pemodelan dilakukan, dataset terlebih dahulu diperiksa distribusi variabel targetnya (Revenue). Berdasarkan hasil eksplorasi data yang telah dilakukan sebelumnya, diketahui bahwa dataset mengalami ketidakseimbangan kelas (*class imbalance*) yang cukup signifikan. Dari total 12.330 observasi, sebanyak 10.422 data (84,53%) termasuk ke dalam kelas 0 (Revenue = *False*), sedangkan hanya 1.908 data (15,47%) yang termasuk ke dalam kelas 1 (Revenue = *True*). Kondisi ini berpotensi menyebabkan model klasifikasi lebih cenderung memprediksi kelas mayoritas sehingga performa dalam mendeteksi kelas minoritas menjadi tidak optimal.

Untuk mengatasi permasalahan tersebut, diterapkan metode *Synthetic Minority Over-sampling Technique* (SMOTE) yang bekerja dengan cara membangkitkan sampel sintesis baru pada kelas minoritas berdasarkan interpolasi antara titik-titik data yang sudah ada. Penerapan SMOTE ini secara khusus hanya

dilakukan pada data training, sedangkan data testing tetap dijaga pada kondisi distribusi aslinya untuk memastikan evaluasi model bersifat objektif dan terhindar dari risiko kebocoran data (*data leakage*). Hasil penerapan SMOTE terhadap data training disajikan pada tabel 4.() berikut.

Tabel 4.2 Perbandingan Distribusi Data Sebelum dan Setelah SMOTE

Keterangan	Sebelum SMOTE	Setelah SMOTE
Jumlah Sampel Training	9.864 Sampel	16.734 Sampel
Kelas 0 (Tidak Revenue)	84,53%	50,0%
Kelas 1 (Revenue)	15,47%	50,0%

Berdasarkan tabel di atas, dimensi dataset *training* yang semula berjumlah 9.864 sampel mengalami peningkatan menjadi 16.734 sampel setelah SMOTE diterapkan. Distribusi variabel target pada data training kini telah mencapai kondisi seimbang sempurna (*perfectly balanced*), di mana kelas 0 dan kelas 1 masing-masing memiliki jumlah sampel identik sebanyak 8.367 data atau setara 50,0% untuk setiap kelas. Sementara itu, data testing tetap dipertahankan pada ukuran aslinya sebanyak 2.466 sampel untuk keperluan evaluasi yang objektif.

2. Pembagian Data (*Split Data*)

Setelah proses penyeimbangan kelas menggunakan SMOTE selesai dilakukan, data training yang telah di-*resample* kemudian dibagi kembali menggunakan metode *train_test_split* dengan proporsi 80% untuk data *training* dan 20% untuk data *testing*. Proses pembagian ini menggunakan parameter *random_state* = 42 untuk memastikan hasil dapat direproduksi, serta parameter *stratify* untuk menjaga proporsi kelas tetap seimbang pada kedua subset data.

Hasil pembagian data menunjukkan bahwa data training terdiri dari 13.387 sampel dengan 24 fitur prediktor, sedangkan data *testing* terdiri dari 3.347 sampel. Melalui struktur pembagian ini, sebanyak 13.387 baris data disiapkan sebagai basis pembelajaran algoritma untuk mengenali karakteristik dan pola fitur, sedangkan 3.347 baris data lainnya dijaga sebagai data independen untuk menguji keakuratan serta kemampuan generalisasi model dalam memprediksi data baru yang belum pernah dipelajari sebelumnya.

3. Pemodelan Klasifikasi

Pada tahap ini, dilakukan pembangunan model klasifikasi menggunakan lima algoritma *machine learning*, yaitu: Random Forest, Logistic Regression, XGBoost, Decision Tree, dan Support Vector Machine (SVM). Seluruh model dilatih menggunakan data training yang sama dan dievaluasi pada data *testing* yang sama pula agar perbandingan performa antar model bersifat adil dan konsisten.

a. Random Forest Classifier

Model Random Forest merupakan algoritma ensemble berbasis bagging yang membangun sejumlah pohon keputusan secara paralel kemudian menggabungkan prediksinya melalui mekanisme *majority voting*. Model ini diinisialisasi menggunakan parameter `random_state = 42` untuk menjamin reproduktibilitas hasil.

Hasil evaluasi model menunjukkan bahwa Random Forest mencatatkan akurasi data training yang nyaris sempurna sebesar 99,99%, dengan nilai *precision*, *recall*, dan *F1-score* mencapai 100% untuk kedua kelas. Pada data *testing*, model mempertahankan akurasi yang tinggi sebesar 92,92% dengan nilai F1-score sebesar 93% secara konsisten untuk kedua kelas. Secara khusus, nilai *recall* pada kelas 1 (*Revenue = True*) berhasil mencapai 94,92%, menegaskan bahwa model memiliki sensitivitas yang sangat tinggi dalam mendeteksi pengunjung yang berpotensi melakukan transaksi.

b. Logistic Regression

Model Random Forest merupakan algoritma ensemble berbasis bagging yang membangun sejumlah pohon keputusan secara paralel kemudian menggabungkan prediksinya melalui mekanisme *majority voting*. Model ini diinisialisasi menggunakan parameter `random_state = 42` untuk menjamin reproduktibilitas hasil.

Hasil evaluasi model menunjukkan bahwa Random Forest mencatatkan akurasi data training yang nyaris sempurna sebesar 99,99%, dengan nilai *precision*, *recall*, dan *F1-score* mencapai 100% untuk kedua kelas. Pada data *testing*, model mempertahankan akurasi yang tinggi sebesar 92,92% dengan nilai F1-score sebesar 93% secara konsisten untuk kedua kelas. Secara khusus,

nilai recall pada kelas 1 (Revenue = *True*) berhasil mencapai 94,92%, menegaskan bahwa model memiliki sensitivitas yang sangat tinggi dalam mendeteksi pengunjung yang berpotensi melakukan transaksi.

c. XGBoost Classifier

Model XGBoost (Extreme Gradient Boosting) merupakan algoritma *ensemble boosting* yang membangun pohon keputusan secara sekuensial, di mana setiap pohon baru berupaya mengoreksi kesalahan yang dibuat oleh pohon sebelumnya. Model ini diinisialisasi dengan parameter `eval_metric = 'logloss'` dan `random_state = 42`.

Pada tahap pelatihan, XGBoost mencatatkan akurasi sebesar 98,36% dengan F1-score 98% untuk kedua kelas, membuktikan kemampuan algoritma *gradient boosting* dalam mengoptimalkan batas keputusan secara mendalam. Saat diuji pada data *testing*, model mempertahankan akurasi yang solid sebesar 92,38% dengan F1-score stabil di angka 92% untuk kedua kelas. Nilai recall pada kelas 1 mencapai 93%, menunjukkan sensitivitas model yang andal. Selisih antara akurasi *training* dan *testing* sebesar 5,98% menunjukkan adanya sedikit *overfitting*, tetapi lebih terkontrol dibandingkan Random Forest maupun Decision Tree.

d. Decision Tree

Model Decision Tree merupakan algoritma klasifikasi yang membangun struktur pohon keputusan tunggal dengan membagi data secara rekursif berdasarkan fitur yang paling informatif. Model ini diinisialisasi menggunakan parameter `random_state = 42` tanpa batasan kedalaman pohon.

Hasil evaluasi menunjukkan bahwa Decision Tree mengalami *overfitting* yang cukup kuat, yang ditandai oleh akurasi data *training* yang nyaris sempurna sebesar 99,99%, sementara pada data *testing* hanya mencapai 89,57%. Kesenjangan ini disebabkan oleh sifat dasar pohon keputusan tunggal yang cenderung membuat percabangan sangat spesifik terhadap data latih. Meskipun demikian, *Classification Report* pada data *testing* menunjukkan performa yang cukup seimbang dengan nilai F1-score dan recall masing-masing sebesar 90% untuk kedua kelas.

e. Support Vector Machine (SVM)

Model Support Vector Machine (SVM) merupakan algoritma yang bekerja dengan mencari hyperplane optimal untuk memisahkan kelas data dalam ruang fitur berdimensi tinggi. Model SVM diinisialisasi dengan kernel Radial Basis Function (RBF) sebagai kernel bawaan dan parameter `random_state = 42`.

Di antara seluruh algoritma yang diuji, SVM mencatatkan performa yang paling rendah dan paling tidak optimal. Pada tahap pelatihan, model hanya memperoleh akurasi sebesar 60,75%, sedangkan pada data *testing* mencapai 61,10%. Kemiripan nilai akurasi antara *training* dan *testing* mengindikasikan tidak adanya *overfitting*, namun model berada dalam kondisi *underfitting* karena gagal menangkap pola data secara mendalam. Nilai *precision*, *recall*, dan *F1-score* yang stagnan di kisaran 60–62% untuk kedua kelas menunjukkan bahwa SVM dengan konfigurasi *default* kurang mampu memisahkan karakteristik antar kelas pada dataset yang digunakan, kemungkinan karena kernel RBF bawaan belum teroptimasi untuk karakteristik dan skala data ini.

4. Perbandingan Performa Model

Ringkasan perbandingan performa seluruh model klasifikasi yang dibangun disajikan pada tabel berikut.

Tabel 4.3 Perbandingan Matriks Evaluasi Lima Model Klasifikasi

Model	Data	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1-Score</i>
Random Forest	<i>Training</i>	99,99%	100%	100%	100%
	<i>Testing</i>	92,92%	93%	93%	93%
Logistic Regression	<i>Training</i>	87,63%	88%	88%	88%
	<i>Testing</i>	87,15%	88%	87%	87%
XGBoost	<i>Training</i>	98,36%	98%	98%	98%
	<i>Testing</i>	92,38%	92%	92%	92%
Decision Tree	<i>Training</i>	99,99%	100%	100%	100%
	<i>Testing</i>	89,57%	90%	90%	90%
SVM	<i>Training</i>	60,75%	61%	61%	61%

Model	Data	Accuracy	Precision	Recall	F1-Score
	Testing	61,10%	61%	61%	61%

Berdasarkan hasil perbandingan pada Tabel 4.() di atas, terdapat perbedaan performa yang cukup signifikan antar model. Random Forest unggul sebagai model terbaik dengan akurasi data *testing* tertinggi sebesar 92,92%, diikuti oleh XGBoost (92,38%), Decision Tree (89,57%), Logistic Regression (87,15%), dan SVM yang berada di posisi terendah (61,10%).

Keunggulan Random Forest dibandingkan model lainnya dapat dijelaskan melalui mekanisme *ensemble bagging*-nya yang mampu mengurangi varians prediksi secara signifikan dengan cara menggabungkan output dari ratusan pohon keputusan yang dibangun secara independen. Berbeda dengan Decision Tree tunggal yang rentan terhadap *overfitting* akibat percabangan yang terlalu spesifik terhadap data latih, mekanisme *averaging* pada Random Forest secara efektif meredam kelemahan tersebut sehingga menghasilkan prediksi yang lebih stabil dan generalisatif pada data baru.

Perlu dicatat pula bahwa selain akurasi keseluruhan, nilai recall pada kelas 1 (*Revenue = True*) merupakan metrik yang paling krusial dalam konteks permasalahan ini. Nilai *recall* yang tinggi berarti *model* mampu mendeteksi sebagian besar pengunjung yang benar-benar akan melakukan pembelian, sehingga meminimalkan risiko perusahaan kehilangan momen untuk memberikan intervensi pemasaran yang tepat sasaran. Dalam hal ini, Random Forest kembali unggul dengan nilai *recall* kelas 1 pada data *testing* sebesar 94,92%, lebih tinggi dibandingkan XGBoost (93%), Decision Tree (90%), Logistic Regression (81%), dan SVM (63%).

Berdasarkan seluruh *pertimbangan* tersebut, Random Forest Classifier dipilih sebagai model terbaik untuk dikembangkan lebih lanjut pada tahap optimasi *hyperparameter*. Model ini tidak hanya unggul dari segi akurasi global dan nilai recall kelas target, tetapi juga menunjukkan performa yang sangat seimbang (*well-balanced*) antara kedua kelas dengan nilai *F1-score* yang konsisten di angka 93%.

C. Analisis *Overfitting* dan *Underfitting*

Selain membandingkan performa model menggunakan metrik evaluasi, penelitian ini juga menganalisis indikasi *overfitting* dan *underfitting* dengan membandingkan

performa model pada data training dan data testing. Analisis ini bertujuan mengetahui kemampuan generalisasi masing-masing algoritma, yaitu kemampuan model mempertahankan performanya ketika dihadapkan pada data baru yang belum pernah digunakan selama proses pelatihan.

Random Forest mencatat akurasi 99,99% pada data training dan 92,92% pada data testing, selisih sebesar 7,07%. Penurunan ini tergolong kecil dan akurasi pada data testing tetap tinggi, sehingga model ini hanya mengalami *overfitting* ringan dengan kemampuan generalisasi yang baik. Kondisi tersebut dipengaruhi oleh mekanisme bagging, yang menggabungkan banyak pohon keputusan sehingga mengurangi varians model dan menghasilkan prediksi yang lebih stabil.

Model Logistic Regression menghasilkan akurasi training 87,63% dan akurasi testing 87,15%, selisih hanya 0,48%. Perbedaan yang kecil ini menunjukkan model tidak mengalami *overfitting* maupun *underfitting*. Namun, karena hubungan antarvariabel pada dataset cukup kompleks, kemampuan prediksi Logistic Regression tetap berada di bawah Random Forest dan XGBoost.

XGBoost memperoleh akurasi 98,36% pada data training dan 92,38% pada data testing, selisih 5,98%, indikasi *overfitting* ringan yang masih menyisakan kemampuan generalisasi yang baik. Capaian ini menunjukkan mekanisme boosting pada XGBoost efektif mempelajari pola data tanpa banyak mengorbankan performa pada data yang belum pernah dilihat sebelumnya.

Berbeda dari ketiga model di atas, Decision Tree menunjukkan indikasi *overfitting* yang paling jelas. Akurasi training mencapai 99,99%, sedangkan akurasi testing hanya 89,57%, selisih 10,42%, tanda bahwa model terlalu menyesuaikan diri dengan data training sehingga kemampuan generalisasinya menurun. Walau demikian, akurasi testing yang mendekati 90% berarti model masih memberikan prediksi yang cukup baik, hanya saja tidak sebaik Random Forest maupun XGBoost.

Sementara itu, Support Vector Machine (SVM) memperoleh akurasi training 60,75% dan akurasi testing 61,10%, dua nilai yang relatif rendah dan hampir setara. Pola ini bukan *overfitting*, melainkan *underfitting*: model belum mampu mempelajari pola penting dalam data sehingga performanya rendah baik pada data training maupun data testing. Penyebabnya diduga penggunaan parameter bawaan (default), yang

membuat model belum mampu membentuk batas keputusan (decision boundary) yang optimal.

Random Forest dan XGBoost hanya mengalami *overfitting* ringan dengan generalisasi yang tetap baik, Decision Tree mengalami *overfitting* yang lebih nyata, sedangkan SVM mengalami *underfitting*. Atas dasar ini, Random Forest dipilih sebagai model terbaik karena memberikan keseimbangan antara akurasi tinggi, kemampuan generalisasi yang baik, dan performa yang stabil pada data training maupun testing, sehingga model ini mampu mempelajari pola pada data tanpa kehilangan kemampuan memprediksi data baru.

D. Hasil Hyperparameter Tuning

Sebelum dilakukan proses optimasi hyperparameter, model Random Forest telah menunjukkan performa yang baik dibandingkan dengan model lainnya. Pada data testing, model Random Forest memperoleh akurasi sebesar 92,92%, menjadikannya model terbaik di antara lima algoritma yang dibandingkan, yaitu Logistic Regression (87,15%), Decision Tree (89,57%), XGBoost (92,38%), dan SVM (61,10%).

Model awal (baseline) ini menunjukkan performa yang sangat baik dalam mendeteksi pembeli potensial, dengan nilai recall pada kelas Revenue = 1 mencapai 94,92%. Artinya, dari setiap 100 pengunjung yang benar-benar melakukan pembelian, model mampu mengenali sekitar 95 di antaranya dengan tepat. Dalam konteks e-commerce, kemampuan ini sangat krusial kesalahan model dalam menebak pembeli asli sebagai "tidak membeli" (*false negative*) akan membuat perusahaan kehilangan momen untuk memberikan intervensi pemasaran yang tepat sasaran.

Namun di sisi lain, model ini menunjukkan gejala *overfitting* yang cukup kuat. Hal ini terlihat dari akurasi data latih (*training*) yang nyaris sempurna sebesar 99,99%, sementara akurasi data uji (*testing*) hanya mencapai 92,92%. Gap sebesar kurang lebih 7,07% ini menandakan bahwa model belum optimal dalam beradaptasi dengan data baru yang belum pernah dilihat sebelumnya. Oleh karena itu, langkah selanjutnya adalah melakukan optimasi hyperparameter untuk mencari pengaturan yang lebih seimbang, sehingga model tidak hanya pintar saat latihan, tetapi juga akurat saat diterapkan pada kondisi nyata.

1. Hasil Optimasi Hyperparameter Random Forest

Proses optimasi *hyperparameter* dalam penelitian ini menggunakan metode RandomizedSearchCV dari *library* Scikit-Learn. Berbeda dengan GridSearchCV yang memeriksa seluruh kombinasi secara menyeluruh, RandomizedSearchCV bekerja dengan cara mengambil sampel kombinasi parameter secara acak. Pendekatan ini dipilih karena jauh lebih hemat waktu dan tenaga komputer, terutama ketika pilihan parameter yang ingin diuji sangat banyak. Dalam optimasi algoritma Random Forest ini, ada empat parameter utama yang disesuaikan: jumlah pohon yang dibangun (*n_estimators*) dengan rentang 100 hingga 500 pohon; batas kedalaman maksimal pohon (*max_depth*) mulai dari tanpa batasan hingga kedalaman 50; jumlah minimal sampel untuk membagi cabang (*min_samples_split*) antara 2 hingga 20; serta jumlah minimal sampel yang harus ada di setiap daun akhir (*min_samples_leaf*) antara 1 hingga 10. Pengaturan pada tiga parameter terakhir sengaja diuji untuk mengontrol agar pohon keputusan tidak tumbuh terlalu detail sehingga bisa mengatasi masalah *overfitting* yang ditemukan sebelumnya.

Untuk menemukan kombinasi terbaik, proses RandomizedSearchCV ini diatur melakukan 100 kali eksperimen acak. Pada setiap eksperimen, model diuji menggunakan metode *5-fold cross-validation* (uji silang 5 tahap), sehingga total ada 500 kali proses pelatihan model yang berjalan. Kualitas dari setiap kombinasi parameter tersebut dinilai berdasarkan tingkat akurasi yang dihasilkan. Langkah terakhir yang dilakukan agar proses pencarian ini tidak memakan waktu lama dilakukan dengan cara seluruh tahapan dijalankan secara paralel dengan memanfaatkan seluruh inti komputer yang tersedia (*n_jobs=-1*).

2. Hasil Parameter Optimal

Setelah seluruh proses RandomizedSearchCV selesai dijalankan, diperoleh kombinasi *hyperparameter* terbaik pada tabel 4.4 berikut:

Tabel 4.4 Tabel Kombinasi *Hyperparameter*

Parameter	Nilai
<i>m_estimators</i>	360
<i>max_depth</i>	30
<i>min_samples_split</i>	3

min_samples_leaf	1
------------------	---

Kombinasi parameter ini menghasilkan akurasi *cross-validation* terbaik sebesar 93,37% pada data latih. Berbeda dengan model baseline yang membiarkan pohon tumbuh tanpa batas ($\text{max_depth} = \text{None}$), proses pencarian justru memilih $\text{max_depth}=30$ sebagai nilai optimal. Hal ini menunjukkan bahwa pembatasan kedalaman pohon secara moderat meskipun tidak ekstrem, tetapi tetap memberikan kontribusi pada peningkatan skor validasi silang dibandingkan tanpa batasan sama sekali.

Di sisi lain, terpilihnya $n_estimators = 360$ menunjukkan bahwa penambahan jumlah pohon keputusan berkontribusi positif terhadap stabilitas performa model. Fenomena ini konsisten dengan karakteristik teoretis algoritma Random Forest, di mana peningkatan jumlah pohon dalam ensemble menghasilkan prediksi yang lebih stabil karena mekanisme agregasi (*averaging*) dari banyak pohon mampu mereduksi varians prediksi secara keseluruhan.

3. Evaluasi Model Setelah Optimasi

Setelah melalui proses *hyperparameter tuning* menggunakan metode RandomizedSearchCV, model Random Forest dengan parameter optimal kemudian dievaluasi menggunakan data pengujian (*test set*) untuk mengukur kemampuan generalisasinya pada data baru. Sebagai pembandingan stabilitas, evaluasi juga dilakukan pada data latih (*training set*).

Secara umum, hasil pengujian menunjukkan performa klasifikasi yang sangat kuat dan seimbang di kedua kelas target, yaitu Kelas 0 (pengunjung yang tidak melakukan pembelian) dan Kelas 1 (pengunjung yang melakukan pembelian). Distribusi metrik evaluasi model setelah proses optimasi dapat dilihat pada Tabel 4.5.

Tabel 4.5 Metrik Evaluasi Model Random Forest Setelah *Tuning*

Kumpulan Data (Set)	Kelas / Metrik	Precision	Recall	F1-Score	Akurasi
Data Pengujian (<i>Test Set</i>)	Kelas 0 (Tidak Membeli)	95,00%	91,00%	93,00%	92,86%

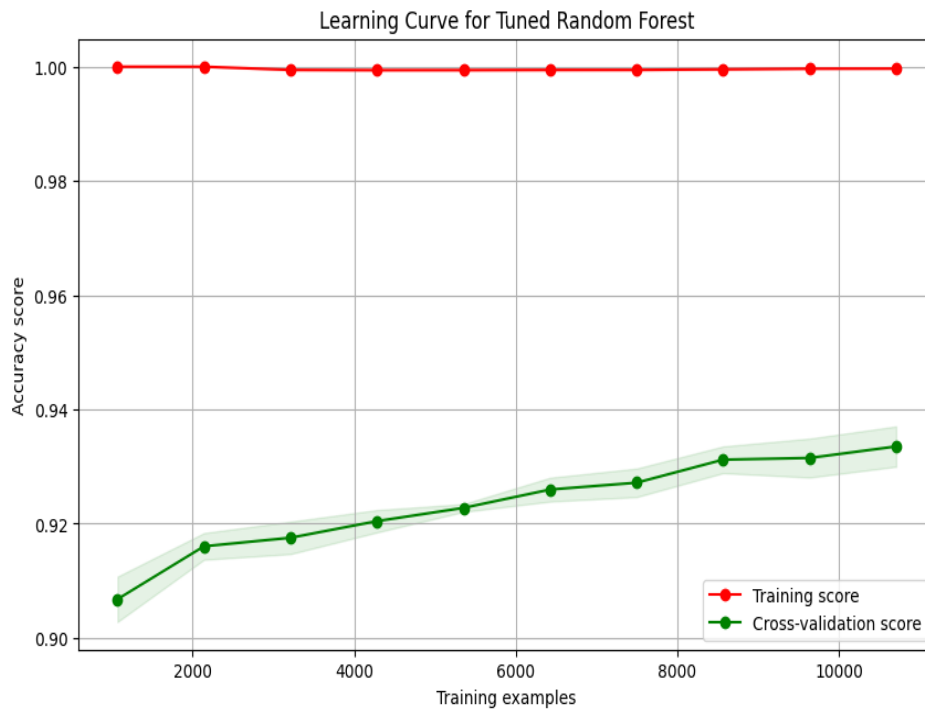
Kumpulan Data (Set)	Kelas / Metrik	Precision	Recall	F1-Score	Akurasi
	Kelas 1 (Membeli)	91,00%	95,04%	93,00%	92,86%
	<i>Macro Average</i>	93,00%	93,00%	93,00%	92,86%
	<i>Weighted Average</i>	93,00%	93,00%	93,00%	92,86%
Data Latihan (Training Set)	Performa Keseluruhan	100,00%	100,00%	100,00%	99,95%

Berdasarkan data yang disajikan pada Tabel 4.5, model Random Forest setelah tuning menunjukkan performa riil yang solid pada data pengujian dengan tingkat akurasi mencapai 92,86%. Konsistensi performa ini dipertegas oleh nilai *macro average* dan *weighted average* yang stabil di angka 93,00%, mengindikasikan bahwa model memiliki kemampuan klasifikasi yang seimbang dan tidak bias terhadap salah satu kelas target. Pada Kelas 1 (pengunjung yang melakukan pembelian), model mencatatkan nilai *recall* sebesar 95,04%, yang menunjukkan sensitivitas sangat tinggi dalam mendeteksi pengunjung yang berpotensi melakukan transaksi. Nilai ini bahkan mengalami peningkatan tipis dibandingkan model awal sebelum optimasi (94,92%), sebuah pencapaian krusial dalam meminimalkan risiko *false negative* atau kegagalan dalam mendeteksi calon pembeli. Kendati demikian, evaluasi pada data latih (*training set*) yang mencatatkan akurasi hampir sempurna sebesar 99,95% mengonfirmasi adanya gejala *overfitting*. Kesenjangan (*gap*) performa yang cukup signifikan antara data latih dan data pengujian ini menunjukkan bahwa model masih cenderung menghafal pola data secara berlebihan, meskipun telah melalui proses pembatasan parameter via *hyperparameter tuning*.

4. Analisis *Learning Curve*

Untuk mendalami perilaku model optimal secara lebih komprehensif, dilakukan analisis kurva pembelajaran (*learning curve*) menggunakan metode 5-fold cross-validation dengan 10 variasi ukuran data training (`np.linspace(0.1, 1.0, 10)`). Grafik ini mengilustrasikan perbandingan antara skor pelatihan (*training score*) dan skor validasi silang (*cross-validation score*) berdasarkan variasi

volume data latih yang digunakan. Berikut akan ditampilkan grafik *learning curve* pada Gambar 4.2.



Gambar 4.2 Grafik Learning Curve untuk Tuned Random Forest

Bedasarkan grafik tersebut, terdapat tiga temuan penting yang berkaitan dengan karakteristik model:

- Skor pelatihan (garis merah) secara konsisten berada pada nilai mendekati 1,00 di seluruh variasi ukuran sampel. Kondisi ini mencerminkan kapasitas belajar model yang sangat tinggi, di mana model selalu mampu menghafal detail data latih secara hampir sempurna.
- Skor validasi silang (garis hijau) menunjukkan tren positif yang meningkat secara bertahap seiring bertambahnya volume data, dari sekitar 90,8% pada kisaran 2.000 sampel hingga mencapai kisaran 93,0% pada lebih dari 10.000 sampel. Tren kenaikan yang belum melandai (*plateau*) ini mengindikasikan bahwa model belum mencapai titik jenuh, sehingga penambahan data latih di masa mendatang berpotensi meningkatkan kemampuan generalisasi model lebih jauh.
- Keberadaan celah (*gap*) yang konsisten antara skor pelatihan dan skor validasi silang merupakan indikator klasik dari *overfitting* (*high variance*). Meski celah ini tidak melebar seiring bertambahnya data, ia juga tidak menutup

sepenuhnya. Namun, area bayangan (*confidence interval*) di sekitar garis validasi tampak semakin menyempit seiring bertambahnya data, menunjukkan bahwa varians prediksi model menjadi lebih stabil dan konsisten berkat dukungan volume data yang lebih besar.

5. Perbandingan Model Awal dan Model Hasil Tuning

Untuk mengevaluasi dampak dari proses optimasi parameter, ringkasan perbandingan performa model Random Forest sebelum dan sesudah tuning disajikan pada tabel 4.6 berikut.

Tabel 4.6 Perbandingan Matriks Evaluasi Sebelum dan Setelah Tuning

No	Data	Perlakuan	Accuracy	Recall	Precision	F1-Score
1	Training Set	Sebelum Tuning	0,9999	1	1	1
	Test Set		0,9319	0,93	0,93	0,93
2	Training Set	Setelah Tuning	0,9995	1	1	1
	Test Set		0,9286	0,93	0,93	0,93

Perbandingan nilai metrik evaluasi pada tabel di atas menunjukkan bahwa terdapat perbedaan hasil matriks evaluasi sebelum dan setelah dilakukan tuning. Secara numerik, proses optimasi menggunakan `RandomizedSearchCV` tidak memberikan peningkatan performa yang berarti, melainkan memicu sedikit penurunan marjinal pada akurasi data pengujian (*test set*) sebesar 0,33%, yaitu dari 0,9319 menjadi 0,9286. Sementara itu, nilai macro average untuk metrik recall, precision, dan F1-score pada data pengujian secara konsisten bertahan di angka 0,93 untuk kedua kondisi model baik sebelum maupun sesudah tuning.

Penurunan tipis pada akurasi tersebut dapat diinterpretasikan melalui dua perspektif utama. Dari sudut pandang optimasi, fenomena ini membuktikan bahwa konfigurasi default algoritma Random Forest sudah selaras secara optimal dengan karakteristik dataset yang digunakan dalam penelitian ini. Karakteristik bawaan Random Forest terbukti memiliki robustisitas yang tinggi sehingga ruang peningkatan performa melalui manipulasi kombinasi hyperparameter menjadi

sangat terbatas. Dari perspektif stabilitas model, perbedaan akurasi yang berada di bawah ambang 0,5% tersebut secara statistik maupun praktis tidak signifikan. Selisih minor ini kemungkinan besar dipicu oleh variabilitas stokastik pada saat proses pembagian (*splitting*) data sehingga kedua variasi model dapat dikategorikan berada pada tingkat performa yang setara.

Di sisi lain, proses tuning ini belum mampu memitigasi kendala *overfitting* secara signifikan. Celah (*gap*) performa antara akurasi data latih dan data pengujian tetap bertahan pada rentang 6,8% hingga 7,1% masing-masing 6,8% pada model awal (99,99% sampai 93,19%) dan 7,1% pada model setelah tuning (99,95% sampai 92,86%). Hal ini terjadi karena meskipun parameter optimal hasil pencarian sudah membatasi *max_depth* pada nilai 30 (bukan tanpa batas seperti default), batasan tersebut masih relatif longgar sehingga pohon keputusan tetap dapat tumbuh cukup dalam dan kompleks. Dengan demikian, temuan ini menegaskan bahwa untuk menekan derajat *overfitting* secara lebih efektif pada dataset ini, diperlukan pendekatan regularisasi struktural yang lebih ketat (misalnya membatasi *max_depth* pada nilai yang lebih kecil, atau menaikkan ambang *min_samples_leaf*) atau intervensi langsung pada arsitektur data, alih-alih hanya mengandalkan pencarian parameter berbasis ruang acak.

BAB V

PENUTUP

A. Kesimpulan

Berdasarkan tujuan penelitian yang telah dirumuskan, yaitu untuk menganalisis karakteristik data, mengidentifikasi fitur penting, serta mengimplementasikan dan mengukur performa model *machine learning* terbaik, dapat ditarik kesimpulan sebagai berikut:

1. Karakteristik Data dan Pola Perilaku Pengunjung: Dataset Online Shoppers Intention yang terdiri dari 12.330 sesi kunjungan menunjukkan bahwa mayoritas pengunjung tidak melakukan transaksi (84,53%) dengan distribusi kelas yang tidak seimbang. Analisis eksploratif mengungkapkan bahwa sebagian besar variabel numerik, seperti ProductRelated dan PageValues, terdistribusi miring ke

kanan, yang mengindikasikan bahwa perilaku pengunjung sangat beragam: sebagian besar hanya melakukan eksplorasi singkat, sementara segelintir pengunjung lainnya menunjukkan intensitas aktivitas yang sangat tinggi.

2. Fitur-Fitur Paling Signifikan: Berdasarkan analisis *feature importance*, ditemukan bahwa faktor perilaku pengunjung saat berinteraksi dengan halaman produk merupakan determinan utama dalam memprediksi pembelian. Tiga variabel dengan pengaruh terbesar adalah ProductRelated_Duration (durasi pada halaman produk), ExitRates (tingkat keluar dari halaman), dan ProductRelated (jumlah halaman produk yang dikunjungi). Temuan ini menegaskan bahwa semakin lama dan semakin banyak pengunjung mengeksplorasi produk, serta semakin rendah tingkat keluar dari situs, maka semakin besar peluang terjadinya transaksi.

3. Implementasi dan Performa Model Terbaik

Dari lima algoritma yang diuji (Random Forest, Logistic Regression, XGBoost, Decision Tree, dan SVM), Random Forest Classifier terpilih sebagai model terbaik. Model ini mampu menghasilkan akurasi tertinggi pada data uji sebesar 92,92% dengan nilai *Recall* untuk kelas pembeli (Revenue = 1) mencapai 94,92%. Performa ini menunjukkan bahwa model sangat baik dalam mengidentifikasi pengunjung yang benar-benar berpotensi membeli, sehingga sangat relevan untuk kebutuhan bisnis. Meskipun proses optimasi *hyperparameter* telah dilakukan, model Random Forest dengan konfigurasi *default*-nya terbukti sudah sangat optimal dan stabil untuk dataset ini.

B. Saran

Berdasarkan kesimpulan di atas, terdapat beberapa saran yang dapat diberikan untuk pengembangan lebih lanjut, baik dari sisi akademis maupun implementasi bisnis:

1. Saran untuk Pengembangan Model

- Kurangi *overfitting*: Untuk model selanjutnya, coba perketat parameter seperti `max_depth` (kedalaman pohon) atau perbanyak `min_samples_leaf` (sampel minimum di daun) agar model lebih sederhana dan tidak terlalu "menghafal" data latih.

- Coba Algoritma Lain: Karena SVM kurang optimal, eksplorasi algoritma yang lebih modern seperti *Deep Learning* atau ensemble lainnya (misal: Stacking) bisa menjadi alternatif untuk meningkatkan akurasi.
 - Tambahkan Data Eksternal: Model akan lebih akurat jika ditambahkan variabel dari luar, seperti data demografi pelanggan atau riwayat kunjungan sebelumnya, untuk menangkap konteks pengunjung secara lebih lengkap.
2. Saran untuk Implementasi Bisnis
- Promosi *Real-Time*: Karena durasi dan jumlah halaman produk adalah indikator kuat, manfaatkan model untuk memberikan diskon atau promo instan ketika sistem mendeteksi pengunjung sedang aktif melihat produk dalam waktu lama, guna mendorong pembelian segera.
 - Perbaiki Tampilan *Website* (UI/UX): Karena BounceRates dan ExitRates menurunkan peluang pembelian, fokuslah untuk mengidentifikasi dan memperbaiki halaman yang sering ditinggalkan pengunjung agar mereka betah berlama-lama di situs.
 - Segmentasi Pelanggan: Integrasikan model ke dalam sistem CRM untuk mengelompokkan pelanggan berdasarkan skor probabilitas pembelian. Dengan begitu, kampanye pemasaran bisa lebih tepat sasaran pada segmen yang paling berpotensi membeli.

LAMPIRAN

Lampiran 1. *Link History Artificial Intelligence*

[Link Gpt](#)

[Link Gemini](#)

Lampiran 2. *Link Repisitory*

[Link Github](#)

Lampiran 3. *Link* Google Collab

[Link Collab](#)